

Transformers II: Construyendo el Modelo

De bloques a arquitectura completa

Cesar Garcia

2025

- **Sesión anterior:** mecanismo de atención
- **Hoy:** construir un transformer completo
- **Enfoque:** hands-on, desde cero
- **Meta:** entrenar un modelo real

Hoy programamos más que hablamos

Bloques que ya conocemos

- Scaled dot-product attention
- Q, K, V proyecciones
- Positional encoding
- Máscaras causales

Hoy: los ensamblamos en un modelo completo

Multi-Head Attention

Implementación manual

```
class MultiHeadAttention(nn.Module):  
    def __init__(self, d_model, num_heads):  
        self.d_k = d_model // num_heads  
        self.W_q = nn.Linear(d_model, d_model)  
        self.W_k = nn.Linear(d_model, d_model)  
        self.W_v = nn.Linear(d_model, d_model)  
        self.W_o = nn.Linear(d_model, d_model)
```

Multi-Head Attention

Proceso

- 1 **Proyectar:** Q, K, V usando W^Q , W^K , W^V
- 2 **Dividir:** en h cabezas (reshape)
- 3 **Atención:** por cabeza independiente
- 4 **Concatenar:** salidas de todas las cabezas
- 5 **Proyectar:** salida final con W^O

Cada paso es diferenciable $\rightarrow$ backprop funciona

Capa posicional

Cada token pasa por una MLP:

```
class FeedForward(nn.Module):  
    def __init__(self, d_model, d_ff):  
        self.fc1 = nn.Linear(d_model, d_ff)  
        self.fc2 = nn.Linear(d_ff, d_model)  
  
    def forward(self, x):  
        return self.fc2(F.relu(self.fc1(x)))
```

Típicamente: $d_{ff} = 4 \times d_{model}$

Por qué son críticas

Sin residuales:

$$X_{out} = \text{Sublayer}(X_{in})$$

Con residuales:

$$X_{out} = X_{in} + \text{Sublayer}(X_{in})$$

Beneficio: gradientes fluyen directamente (evita vanishing)

Estabilidad

Después de cada subcapa:

$$\text{LayerNorm}(X + \text{Sublayer}(X))$$

Normaliza por dimensiones del embedding:

$$\hat{x} = \frac{x - \mu}{\sigma + \epsilon}$$

Resultado: entrenamiento más estable y rápido

Receta

```
class EncoderLayer(nn.Module):  
    def forward(self, x):  
        # 1. Self-attention + residual + norm  
        attn = self.self_attn(x, x, x)  
        x = self.norm1(x + attn)  
  
        # 2. FFN + residual + norm  
        ffn = self.ffn(x)  
        x = self.norm2(x + ffn)  
        return x
```

Apilar N bloques $\rightarrow$ encoder profundo

Arquitectura

- 1 **Input:** tokens $\rightarrow$ embeddings
- 2 **Positional Encoding:** suma
- 3 **$N \times$ Encoder Layers**
- 4 **Output:** predicción (ej: siguiente token)

Tokens $\rightarrow$ Embedding $\rightarrow$ +PE $\rightarrow$ Encoder_1 $\rightarrow$... $\rightarrow$ Encoder

Para decoders

```
def create_causal_mask(seq_len):  
    mask = torch.tril(torch.ones(seq_len, seq_len))  
    return mask  # triangular inferior = 1 (permitido)
```

```
1 0 0 0    ← token 0 solo ve posición 0  
1 1 0 0    ← token 1 ve 0, 1  
1 1 1 0    ← token 2 ve 0, 1, 2  
1 1 1 1    ← token 3 ve 0, 1, 2, 3
```

Character-Level Prediction

Dataset: texto en español

Tarea: predecir siguiente caracter

Arquitectura: pequeño transformer (2 capas, 4 heads)

Resultado: generación de texto

Input: "hola mun"

Output: "do"

Pipeline

- 1 Tokenizar texto (char-level)
- 2 Crear batches de secuencias
- 3 Forward pass con máscara causal
- 4 Loss: cross-entropy
- 5 Backward + optimizer step

Epoch time: ~10 segundos (CPU)

Sampling autoregresivo

```
def generate(model, start, max_len):  
    context = tokenize(start)  
    for _ in range(max_len):  
        logits = model(context)  
        next_token = sample(logits[-1]) # último token  
        context = append(context, next_token)  
    return decode(context)
```

Por qué Escalan los Transformers

Tres claves

- ❶ **Paralelismo total:** no hay dependencias secuenciales
- ❷ **Dependencias globales:** cada token puede atender a todos
- ❸ **Arquitectura homogénea:** mismo bloque repetido

RNN: $O(n)$ steps (secuencial)

Transformer: $O(1)$ steps (paralelo)

Trade-offs

- **Memoria:** $O(n^2)$ por la matriz de atención
- **Complejidad:** $O(n^2 \cdot d)$ por forward pass
- **Secuencias largas:** prohibitivas sin optimizaciones

Soluciones modernas: Sparse attention, Linear attention, etc.

Construcción modular

Los transformers son:

- **Modulares:** bloques independientes y reusables
- **Escalables:** más capas = más capacidad
- **Paralelizables:** GPU-friendly

La arquitectura es simple. El poder viene del escalado.

Bloques construidos

- [X] Multi-head attention
- [X] Feedforward network
- [X] Layer normalization + residuales
- [X] Positional encoding
- [X] Máscara causal
- [X] Encoder completo
- [X] Modelo entrenado

Ahora tienes las herramientas para construir cualquier variante de transformer

Variantes modernas

- **BERT**: encoder bidireccional (masked language modeling)
- **GPT**: decoder causal (autoregresivo)
- **T5**: encoder-decoder (seq2seq)
- **ViT**: transformers para imágenes

Todos usan los mismos bloques que construimos hoy.